Handling missing data

Missing values reflect a data generation process: values are missing for a reason. We need to understand (or at least hypothesize) why data is missing to choose missing value imputation methods that are

- statistically valid
- avoid introducing bias

This tutorial introduces the classical categories of missingness, then reviews practical methods for handling missing values, with particular attention to MICE and KNN imputation.

Classical categories of missingness

A standard way to think about missing data is to classify it by the mechanism that caused the missingness.

MCAR — Missing Completely At Random

Interpretation: Whether a value is missing is unrelated to anything in the dataset.

Example scenarios

- A random software glitch causes some rows to be dropped entirely.
- A sensor occasionally fails independent of environmental conditions.
- Survey forms were lost in the mail randomly.

Practical implication

Under MCAR, analyzing only complete cases does not create systematic bias (but reduces sample size and efficiency), thus

- Complete case analysis is often acceptable (unbiased) but wastes data.
- Imputation tends to work well; many simple methods are “good enough” (MCAR is the least problematic case statistically).

MAR — Missing At Random

Data is missing at random when the probability of missingness depends on observed variables, but not on the missing value itself once those observed variables are taken into account.

Interpretation: Missingness is “explainable” using variables we actually observed.

Example scenarios

- In a survey, people in certain known groups are less likely to answer income questions.
    - Example: for respondents with observed education level/age group, salary nonresponse differs.
- In simulation experiments, the model crashes for certain observed input conditions.
    - Example: for large observed parameter values, the simulation becomes unstable, and the output becomes missing.

Practical implication

- Many modern imputation methods, including multiple imputation and MICE, are built around the MAR assumption.
- Under MAR, methods that properly condition on predictors can still be valid. If missingness is independent of the response conditional on the predictors, linear regression remains valid (conceptually: conditioning blocks the dependence).
- Naive imputation may distort relationships depending on the analysis goal.
- But imputation methods can introduce bias, depending on:
    - whether the imputation uses the response variable,
    - whether your analysis depends on correlations/variance (most do),
    - whether uncertainty is propagated.

MNAR — Missing Not At Random

The probability of missingness depends on the missing value itself, even after conditioning on observed data.

Interpretation: Missingness is informative about the missing value.

Example scenarios

- Pollutant sensor detection limit: the sensor only records values above a threshold; low values are "missing."
    - Here missingness depends on the actual (unobserved) pollutant concentration.
- Storm surges only occur in areas below water level
    - The surge measurements are missing in regions that never become inundated; missingness depends on latent flood conditions.

Practical implication

- Under MNAR, missingness itself must be modeled explicitly.

    - For example, censored observations in survival analysis, where the censoring mechanism is part of the model.
    - The modeling process, in general, requires additional domain knowledge and modeling assumption.
- Standard imputation can lead to strong bias unless missingness is handled carefully.

Methods to Handle Missing Values

1. Removing Data

It's often the default choice in many software packages. It may be reasonable when:

- Data is likely MCAR
- Missingness is very limited
- Simplicity is emphasized

Except for removing all rows that contain at least one missing value (listwise deletion), we can use all available observations for each calculation rather than dropping an entire row. It's called Pairwise deletion. For example, we can only use rows where both are observed when computing the correlation between X and Y.

2. Simple Imputation with a Constant

When it works best:

- For categorical data, where “Unknown” is a meaningful category
- As a baseline

3. Mean, Median, or Mode Imputation

- Mean imputation for continuous variables
- Median imputation for skewed continuous variables
- Mode imputation for categorical variables

they are common benchmarks

4. Multiple Imputation

Basic idea:

- Generate several imputed versions of the dataset
- Fit the desired analysis on each dataset
- Pool the results across datasets

Multiple imputation is not one specific algorithm. It is a framework. **MICE** is one of the most common ways to perform multiple imputation.

Focus Method 1: MICE (Multiple Imputation by Chained Equations)

It is an iterative imputation method that fills in missing values variable by variable.

The reason why it's called "chained equations"

- first impute one variable,
- then use that updated dataset to impute the next,
- and keep cycling until the imputations stabilize.

MICE is a strong choice when:

- several variables have missing values,
- variables are correlated,
- you want a statistically principled approach,
- and your goal includes inference, not just prediction.

Python functions

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import miceforest as mf

# 1. Create kernel
kernel = mf.ImputationKernel(
    data=df,
    random_state=42
)

# 2. Run MICE
kernel.mice(iterations=5)

# 3. Extract one completed dataset
df_mice = kernel.complete_data(dataset=0)

# 4. Distribution plot for imputed variables
kernel.plot_imputed_distributions()
plt.show()
```

Focus Method 2: KNN Imputation

KNN imputation uses the **k nearest neighbors** of an observation to estimate its missing values.

- For numeric variables, the imputed value is often the average of the neighbors' values.
- For categorical variables, it may be the most common category among neighbors.

KNN is often useful when:

- the dataset is moderate in size (not high dimentional),
- similar observations are meaningful,
- the data has local structure,
- and the goal is predictive performance rather than formal statistical inference.

Python functions

```
from sklearn.impute import KNNImputer

# Optional: scale before KNN because distances matter
scaler = StandardScaler()
df_scaled = pd.DataFrame(
    scaler.fit_transform(df),
    columns=df.columns
)

# KNN imputation
imputer = KNNImputer(n_neighbors=3, weights="uniform")
df_knn_scaled = pd.DataFrame(
    imputer.fit_transform(df_scaled),
    columns=df.columns
)

# Transform back to original scale
df_knn = pd.DataFrame(
    scaler.inverse_transform(df_knn_scaled),
    columns=df.columns
)
```

Imputation experiment

We impute missing values for column `ActualElapsedTime` from `airline_2019.csv` using:

- constant imputation
- MICE (iterative imputer with posterior sampling; PMM if `miceforest` is available)
- k-nearest neighbors (KNN) imputation

We compare methods using:

- RMSE and correlation on a held-out missingness mask (numeric columns only)

```
# import required packages
!pip install miceforest
import pandas as pd
import missingno as msno
import miceforest as mf
import plotnine
from sklearn.compose import ColumnTransformer
from sklearn.impute import KNNImputer, SimpleImputer
```

```
Collecting miceforest
  Downloading miceforest-6.0.5-py3-none-any.whl.metadata (34 kB)
Requirement already satisfied: lightgbm>=4.1.0 in /usr/local/lib/pytho
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist
Requirement already satisfied: pandas>=2.1.0 in /usr/local/lib/python3
Requirement already satisfied: pyarrow>=6.0.1 in /usr/local/lib/python
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/li
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/d
Downloading miceforest-6.0.5-py3-none-any.whl (38 kB)
Installing collected packages: miceforest
Successfully installed miceforest-6.0.5
```

```
## Change the file location to your saved location
FILE_LOCATION = 'airline_2019.csv'
```

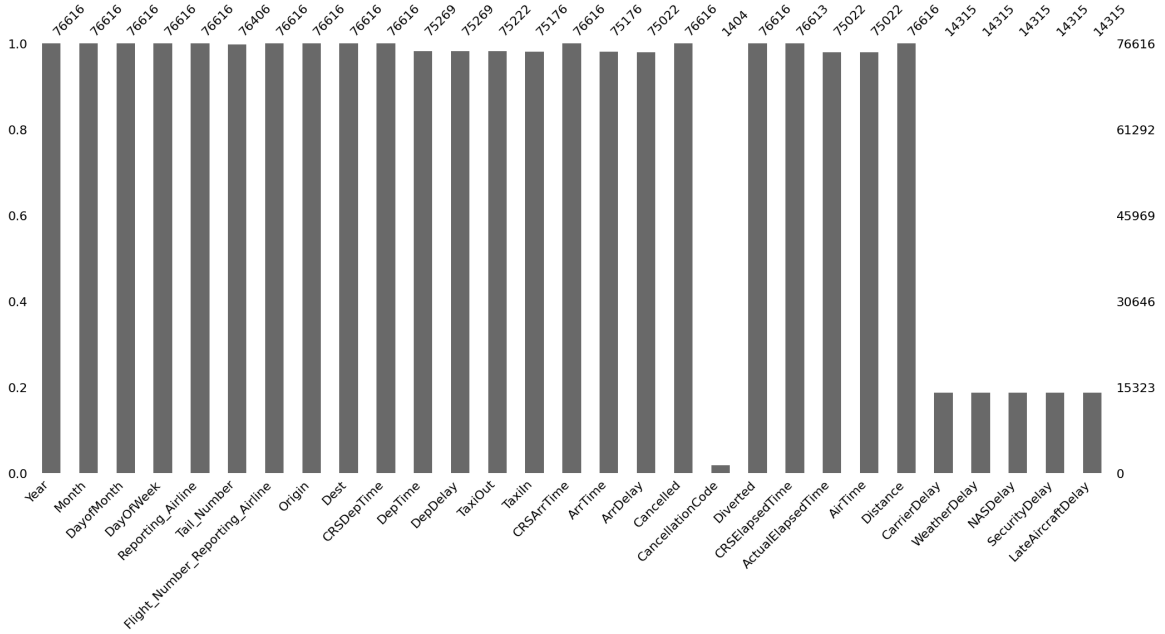
```
## We will use the following subset of columns
USE_COLS = [
    "Year",
    "Month",
    "DayofMonth",
    "DayOfWeek",
    "DepTime",
    "CRSDepTime",
    "ArrTime",
    "CRSArrTime",
    "Reporting_Airline",
    "Flight_Number_Reporting_Airline",
    "Tail_Number",
    "ActualElapsedTime",
    "CRSElapsedTime",
    "AirTime",
    "ArrDelay",
    "DepDelay",
    "Origin",
    "Dest",
    "Distance",
```

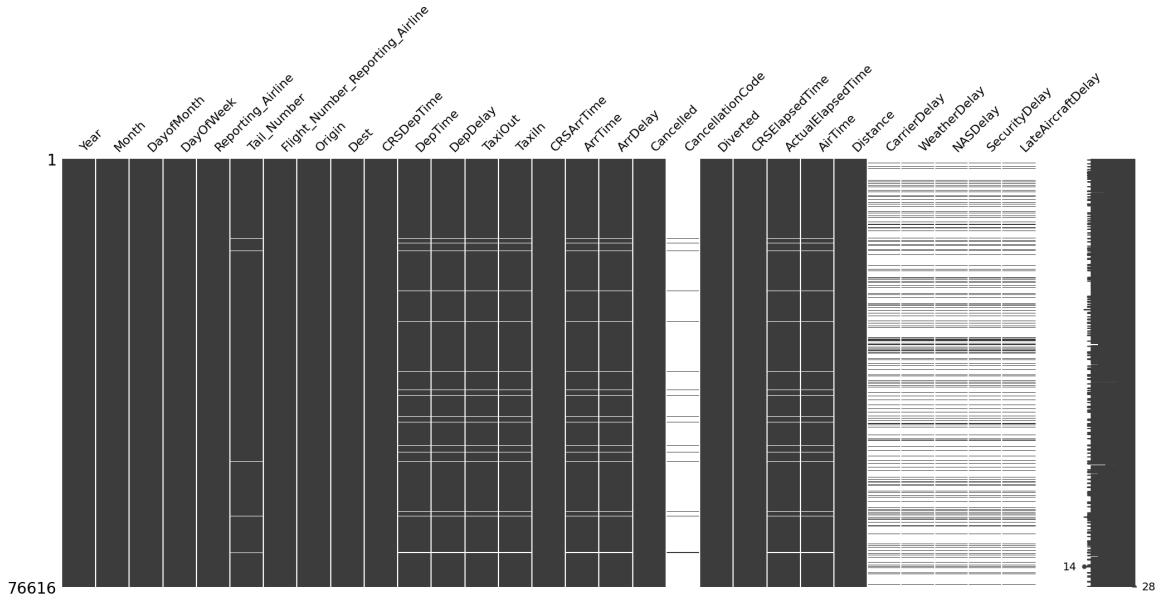
```
        "TaxiIn",
        "TaxiOut",
        "Cancelled",
        "CancellationCode",
        "Diverted",
        "CarrierDelay",
        "WeatherDelay",
        "NASDelay",
        "SecurityDelay",
        "LateAircraftDelay",
    ]
```

```
    # read the data
    df_2019 = pd.read_csv(FILE_LOCATION, usecols = USE_COLS)
```

```
    # # visualization of available/missing data with missingno package
    msno.bar(df_2019)
    msno.matrix(df_2019)
```

<Axes: >


76616 76616 76616 76616 76616 76406 76616 76616 76616 76616 75269 75269 75222 75176 76616 75176 75022 76616 1404 76616 76613 75022 75022 76616 14315 14315 14315 14315 14315
1.0
0.8
0.6
0.4
0.2
0.0
76616
61292
45969
30646
15323
0
Year Month DayofMonth DayOfWeek Reporting_Airline Tail_Number Flight_Number_Reporting_Airline Origin Dest CRSDepTime DepTime DepDelay TaxiOut TaxiIn CRSArrTime ArrTime ArrDelay Cancelled CancellationCode Diverted CRSElapsedTime ActualElapsedTime AirTime Distance CarrierDelay WeatherDelay NASDelay SecurityDelay LateAircraftDelay


Year Month DayofMonth DayOfWeek Reporting_Airline Tail_Number Flight_Number_Reporting_Airline Origin Dest CRSDepTime DepTime DepDelay TaxiOut TaxiIn CRSArrTime ArrTime ArrDelay Cancelled CancellationCode Diverted CRSElapsedTime ActualElapsedTime AirTime Distance CarrierDelay WeatherDelay NASDelay SecurityDelay LateAircraftDelay
1
76616
14
28

Build Holdout Mask

It a boolean mask (same shape as your dataframe) that marks a random subset of currently-observed entries to be held out / artificially treated as missing.

This performs the similar function as removing whole column data in class example. This is to enable us to compre imputed and roginal true values and calculated metrics (e.g., RMSE).

```
import numpy as np

target_col = "ActualElapsedTime"
seed = 42
missing_rate = 0.10

rng = np.random.default_rng(seed)
observed_mask = df_2019[target_col].notna()
holdout_mask = (rng.random(len(df_2019)) < missing_rate) & observed_m

df_masked = df_2019.copy()
df_masked.loc[holdout_mask, target_col] = np.nan

holdout_mask.sum()
```

```
np.int64(7399)
```

```
# we define the following function to calculate RMSE of each method
# you don't need to make modifications here

def evaluate_method(label, df_imputed):
    y_true = df_2019.loc[holdout_mask, target_col].astype(float)
    y_pred = df_imputed.loc[holdout_mask, target_col].astype(float)
    valid = y_true.notna() & y_pred.notna()

    if valid.sum() == 0:
        rmse = np.nan
        corr = np.nan
    else:
        rmse = float(np.sqrt(np.mean((y_true[valid] - y_pred[valid]) 
        corr = float(np.corrcoef(y_true[valid], y_pred[valid])[0, 1])

    return {"method": label, "rmse": rmse}
```

Method 1: Simple Imputation with a constant

```
# Constant imputation
constant_fill = df_2019[target_col].median()
df_const = df_masked.copy()
df_const[target_col] = df_const[target_col].fillna(constant_fill)
```

```
df_const[target_col].isna().sum()

# use .describe() methods to get key statistics of the imputed column
df_const[target_col].describe()
```

	ActualElapsedTime
count	76616.000000
mean	134.482562
std	68.336504
min	16.000000
25%	88.000000
50%	119.000000
75%	159.000000
max	665.000000

dtype: float64

```
evaluate_method("Constant", df_const)
```

```
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.
/usr/local/lib/python3.12/dist-packages/numpy/lib/_function_base_impl.
{'method': 'Constant', 'rmse': 75.19771479448166}
```

Method 2: MICE with miceforest

key function

```
kernel = mf.ImputationKernel(
    df_masked[numeric_cols].astype(float),
    datasets=1,
    save_all_iterations=False,
    random_state=seed,
    mean_match_candidates=5,
)
kernel.mice(mice_iters)
df_mice_num = kernel.complete_data(dataset=0)
```

```
# Get a list of columns that are numeric columns
numeric_cols = df_masked.select_dtypes(include='number').columns
```

```
# When building the MICE kernel, only use the dataset with numeric co
mice_iters = 5
df_num = df_masked[numeric_cols].astype(float)
```

```
kernel = mf.ImputationKernel(
    df_num,
    num_datasets=1,
    save_all_iterations_data=False,
    random_state=seed,
    mean_match_candidates=5
)
```

```
# run the MICE algorithm
kernel.mice(mice_iters)

df_mice_num = kernel.complete_data(dataset=0)
```

```
# use .describe() methods to get key statistics of the imputed column
df_mice = df_masked.copy()
df_mice[numeric_cols] = df_mice_num[numeric_cols]

df_mice[target_col].describe()
```

	ActualElapsedTime
count	76616.000000
mean	136.319202
std	72.183975
min	16.000000
25%	84.000000
50%	118.000000
75%	167.000000
max	665.000000

dtype: float64

```
evaluate_method("mice", df_mice)
```

```
{'method': 'mice', 'rmse': 7.444210667979017}
```

Method 3: KNN imputation

Key function

```
knn_imputer = KNNImputer(n_neighbors=knn_k)
df_knn_num = pd.DataFrame(
    knn_imputer.fit_transform(df_masked[numeric_cols].astype(float)),
    columns=numeric_cols,
```

```
    index=df_masked.index,
)
```

```
# impute with KNN methods
knn_k = 5
knn_imputer = KNNImputer(n_neighbors=knn_k)

df_knn_num = pd.DataFrame(
    knn_imputer.fit_transform(df_masked[numeric_cols].astype(float)),
    columns=numeric_cols,
    index=df_masked.index,
)

# Reconstruct full dataframe with KNN-imputed numeric values
df_knn = df_masked.copy()
df_knn[numeric_cols] = df_knn_num[numeric_cols]
```

```
# print the first few lines of the dataframe with imputed values
df_knn.head()
```

	Year	Month	DayofMonth	DayOfWeek	Reporting_Airline	Tail_Number
0	2019.0	6.0	11.0	2.0	9E	N927XJ
1	2019.0	9.0	2.0	1.0	YX	N203JQ
2	2019.0	3.0	27.0	3.0	OH	N582NN
3	2019.0	2.0	13.0	3.0	WN	N491WN
4	2019.0	5.0	8.0	3.0	DL	N922DZ

5 rows × 29 columns

```
# use .describe() methods to get key statistics of the imputed column
df_knn[target_col].describe()
```

```
ActualElapsedTime
```